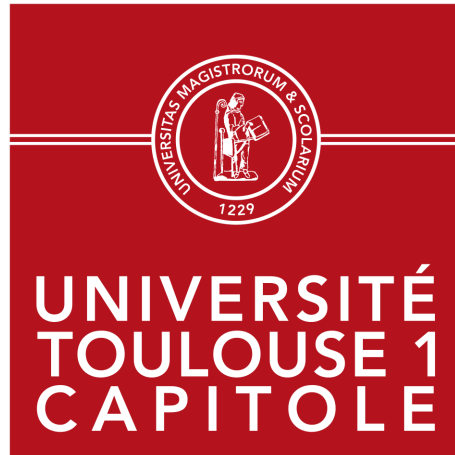




UT1C - M2 MIAGE 2IS
INNOVATIVE INFORMATION SYSTEMS

INTERNSHIP REPORT

Deep Learning For Bathymetry Inversion

Student: Mahmoud AL NAJAR

Academic Supervisor: Dennis G. WILSON

Laboratory Supervisors: Rachid BENSILA, Rafael ALMAR, Erwin
BERGSMA

September, 2019

Acknowledgements

This internship was a great opportunity for me to apply what I had learned during my studies, and to expand my knowledge even further in the field of deep learning. I am truly grateful to have had the chance to work on this project, and to meet and work with different professionals that provided me with their precious help and guidance. I would like to use this as a chance to thank everybody that was, in one way or another, involved in this internship.

First of all, I would like to extend my deepest gratitude to the teaching faculty of the MIAge 2IS program at Université Toulouse 1 Capitole, who gave me the theoretical and practical knowledge that enabled me to achieve this internship. Most notably, I would like to thank Dr. Umberto GRANDI for providing me with his precious guidance and advice during my studies. My academic supervisor Dr. Dennis G WILSON, for his continuous availability and support during my internship, and for providing his valuable input and guidance whenever I needed help.

Furthermore, I would like to thank my supervisors at LEGOS, Mr. Rafael ALMAR, Mr. Rachid BENSILA, and Mr. Erwin BERGSMA, for being very supportive and being always available to provide their feedback and suggestions regarding the physics-based aspects of my work.

Finally, I thank Grégoire THOUMYRE, a colleague at LEGOS that helped me a lot during my internship. As well as Sara PADILLA POLO and Sabriya OULAD, my office-mates that welcomed me and integrated me into the laboratory.

Last but not least, I would like to express my profound gratitude to my family that provided me with their unconditional support throughout my years of study. I would not have been able to reach this point in my academic career without you. Thank you!

Table of Contents

1	Introduction	4
1.1	IRD	4
1.2	LEGOS	4
1.3	The ECOLA Team	5
2	Problem Statement	6
3	Bathymetry Inversion	6
3.1	Sentinel-2 Satellite Constellation	7
3.2	Data Assimilation	9
4	Proposed Method: Deep Learning	10
4.1	Artificial Neural Networks	10
4.1.1	Artificial Neuron: Single Linear Decision Function	10
4.1.2	Multi-layer Perceptrons	11
4.2	The Learning Mechanism	11
4.2.1	Forward-Propagation	12
4.2.2	Loss Functions	12
4.2.3	Back-Propagation & Optimization	13
4.3	ReLU Activation	15
4.4	Deep Learning & Computer Vision	16
4.4.1	Computational Complexity	16
4.4.2	Feature Locality	18
4.4.3	3D Filters for Multi-Channel Inputs	19
4.4.4	Convolutional Neural Networks	19
4.5	Neural Network Architecture Design	20
4.5.1	Very Deep Networks	20
4.5.2	Residual Neural Networks	21
4.5.3	The Auto-Encoder Architecture	22
5	Synthetic Dataset	23
5.1	Constraints	24
5.2	Data Creation	24
5.3	The Processed Data	25
6	Large-Scale 2D Inversion	27
6.1	Base Architecture	27
6.2	Technical Specification	28
6.3	Experiments	29

6.3.1	Loss functions	29
6.3.2	Depth normalization	31
6.4	Results Analysis & Conclusion	32
7	Small-Scale Single-Point Inversion	35
7.1	Base Architecture	35
7.2	Technical Specification	36
7.3	Experiments	37
7.3.1	Initial Models	38
7.3.2	Hyper-Parameter Grid-Search	40
7.4	Results Analysis & Current State	42
8	Discussion & Conclusion	44

1 Introduction

In this section, I will start by introducing the IRD institute that funded my internship, and the LEGOS laboratory that accommodated me throughout the duration of my internship, and I will briefly describe the different domains of work and research currently being done at those institutes. I will also present the ECOLA Team which I was part of during the entirety of my internship.

1.1 IRD

Institut de Recherche pour le Développement (IRD) - The French National Research Institute for Development - is a French public establishment for research in Science and Technology, that operates under the dual authority of the French Ministry of Higher Education, and the French Ministry of Foreign Affairs.

The IRD is considered to have a key role as a French player in the international development agenda. Its work follows a model of equitable scientific research and development in partnership with developing countries, as a way to progress scientifically to reach further sustainable and human development in all areas of the world, and to tackle challenges that face us and the planet as a whole as they arise, including pandemics, climate change, humanitarian and political crises, to name a few.

Research on world development alongside consultancy and training work are some of the IRD's main missions. Their work puts a lot of focus on the relationship between humans and the environment, and it is organised into five different departments as follows:

- Earth and Environment.
- Living Resources.
- Societies and Health.
- Expertise and Consulting.
- Support and Training.

1.2 LEGOS

Laboratoire d'Etudes en Géophysique et Océanographie Spatiales (LEGOS) - Laboratory of Geophysics and Spatial Oceanography Studies - is a Toulouse-based multi-

disciplinary research laboratory. It groups researchers and engineers from four main employing institutes including the national center of spatial studies (CNES), the national center of scientific research (CNRS), the university of Toulouse III - Paul Sabatier (UPS), the French national research institute for development (IRD), among other european and international institutes.

Scientific research at LEGOS can be grouped into four main themes:

- **Glaciology:** Research in this domain is focused on detecting and understanding the physical mechanisms of ice sheets, and their role and affect on the climate system.
- **Hydrology and Satellite-based Geodesy:** Which are concerned with the observation and study of sea level variations and their causes and impacts. As well as the use of satellite observations to monitor and study the global hydrology.
- **Physical Oceanography:** This domain covers a wide range of spatial and temporal scales of study, ranging from gravity waves, to more long-term variations in the state of the global ocean. This domain also incorporates the study of the relationship between these phenomena and the climate of the area under study.
- **Marine Bio-geochemistry:** This area of research can be divided into two main sub-domains:
 1. The study of chemical and isotopic tracers: where the main focus is on studying and measuring the different processes that play a role in the movement of materials in the ocean waters, as well as recreating water mass movements and formation.
 2. Interactions between dynamics and primary production: which refers to the study of the links between human-caused climate change, and changes in ecosystems and marine biogeochemical cycles.

1.3 The ECOLA Team

The observation and modeling of the coastal ocean from regional to nearshore scales team (ECOLA) studies the dynamic relation between the offshore and nearshore areas of the ocean, including external and internal water tides, coastal water surface level, wave-driven coastal development and dynamics, sediment-related processes in estuaries, as well as the variability and effects of climate changes at the regional

scales. Researchers in the ECOLA team do their studies using - mainly - numerical modeling, observation analysis (satellite and in-situ), and data assimilation.

2 Problem Statement

Coastal areas are currently facing environmental and resource problems compounded by population pressure and over-exploitation. The environmental context or extreme events (floods, coastal erosion), combined with population pressure, are a limiting factor for coastal development.

The state and evolution of the physical characteristics of coastal areas over time is an important factor that must be considered for coastal development and planning, but it is a task that is considered expensive, and not always possible, due to the complexity of performing accurate bathymetry scans. Recently, low-cost remote sensing tools such as drones and satellites have emerged as inexpensive data sources, that can be used to estimate and forecast coastal dynamics and features. However, the process of using these data sources to create accurate estimates of bathymetry profiles, also known as bathymetry inversion, is still an ongoing topic of research.

In the following sections, I will present a brief introduction to the current numerical methods used to perform bathymetry inversion. Including the methods currently being developed at LEGOS to do the same task based on satellite imagery, which was the main focus of my internship.

Moving further, I will give a presentation deep learning and the different concepts that make up a modern deep learning model. Finally, I will explain the methods I tested during my internship, along with the results I was able to obtain.

3 Bathymetry Inversion

Currently, bathymetry inversion using remote sensing techniques is one of the major topics of research in the fields of marine physics and coastal oceanography. There are multiple remote sensing data collection tools used including shore-based or drone-mounted video cameras, as well as different types of satellites that provide different types of data at varying intervals.

The current numerical methods of estimating bathymetric profiles from satellite data, which is the focus of my research project, can be classified into two main categories [4].

The first method, **Depth through colour**, estimates depth from *optical imagery* by using the relation between light-beam absorption and the height of water columns. The general rule in this method is that the darker the color is at a certain point in the image, the deeper that point is. While accurate down to tens of centimeters, this method is very sensitive to turbid or aerated waters, due to the somewhat inconsistent changes in color in the resulting image.

The second method is **depth through wave celerity** and it is based on the physical relation between wave celerity and the depth of the underlying bathymetry, where deeper sea-floor leads to faster surface waves. Using this method, local depth can be estimated from *both optical or radar imagery*, using the linear dispersion relation (Equation 1), where c is wave celerity, g represents the gravitational acceleration, and k is the wave number.

$$c^2 = \frac{g}{h} \tanh(kh) \iff h = \frac{\tanh^{-1}(\frac{c^2 k}{g})}{k} \quad (1)$$

This method has been applied to many different types of data collected from different remote sensing tools, ranging from shore-based systems, to airborne and space-borne systems. Theoretically speaking, as long as waves can be observed in the data, this method can be used to estimate depth from shore to intermediate waters. [3, 4].

3.1 Sentinel-2 Satellite Constellation

Some of the main limitations that face traditional in-field measurements and/or current bathymetry inversion applications, are the spatial-concentration and the temporal-sparsity of the data.

Type	Method	Spatial domain	Temporal frequency	Cost
In-situ	Single beam	local	low	low
	Multi-beam	regional	low	high
Remote	Shore-based	local	high	low
	Space-borne	regional to global	high*	high**

Table 1: Comparison of data collection & measurement methods. In-situ relates to field measurements with an echo-sounder (single or multi-beam). In this study "remote" stands for passive sensors such as optical (video) imagery. *depending on the satellite mission, **some satellite data is freely available

From table 1 we can see how using space-borne remote sensing tools would be very useful, especially if there exists a method that can use the collected data to accurately and reliably perform bathymetry inversion, which was the core idea behind my work. For my project, we aimed to make use of data collected by the Sentinel-2 satellite mission [5], which is an earth monitoring constellation composed of two satellites (Figure 1).

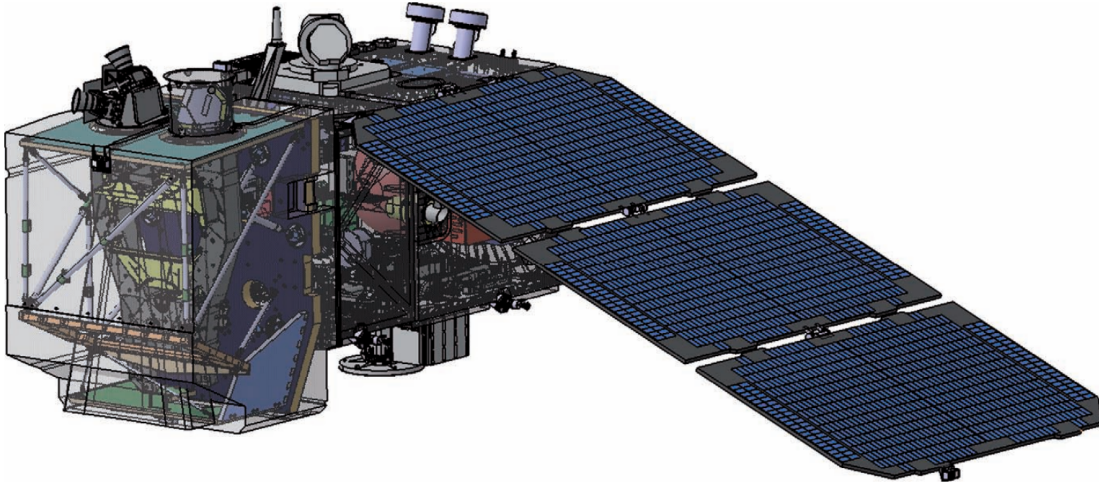


Figure 1: Sentinel-2 Satellite. Source: [5]

The Sentinel-2 mission offers the following capabilities:

- Systematic and global coverage of surfaces from 56°S to 84°N , with a field of view of 290km , covering coastal waters, the Mediterranean and selected calibration sites, e.g. over Antarctica.
- High revisit frequency: every 10 days with one satellite, and every 5 days using both.
- High spatial resolution: 10m , 20m and 60m .
- Multi-spectral information with 13 bands in the VNIR and SWIR parts of the spectrum.

An interesting point about satellite imagery in general is how they incorporate temporal information. A single Sentinel-2 snapshot for example is composed of 10 images (See table 2) collected one wavelength-specific band at a time, creating a time-lag between the different bands which - in a way - *encodes temporal information* into the final snapshots collected.

This temporal information has been used previously in order to detect different

Band	Δt [sec]	Av.wavelength (S2A/S2B) [nm]	Resolution [m]
2	0	492.3	10
8	0.264	832.9	10
3	0.527	559.4	10
4	1.005	664.8	10
5	1.269	704.0	20
6	1.525	739.8	20
7	1.790	781.3	20
8a	2.055	864.4	20
1	2.314	442.5	60
9	2.586	944.1	60

Table 2: Sentinel-2 Detector Bands

wave features including celerity and direction, which are then used for different applications including bathymetry inversion [1] for example.

For my research project, I worked with the highest resolution bands only (10m). However, there is work currently being done at LEGOS to augment the resolution of the other lower-resolution bands [4], giving access to even more temporal information from a single Sentinel-2 snapshot (Further exploration, see section 8).

3.2 Data Assimilation

Data assimilation [6] is a process of combining different sources of information to best estimate the state of a system (bathymetry for example) at a certain point in time, using different mathematical approaches. For example, in the case where there are multiple numerical models, each creating its own estimation, and each having its own error rate, the output of these models can then be combined/assimilated using the current estimates as well as previous estimates and correct observations, to obtain the best/most accurate result possible.

Today, applications of data assimilation generalize to many other fields, including atmospheric chemistry, oceanic biochemistry, glaciology, oil industry, nuclear fusion, medicine, agronomy, etc. As well as many applications, for example:

- The estimation of the trajectory of a system to study its variability (reanalyses).
- The identification of systematic errors in numerical models.

- The optimization of observation network.
- The estimation of unobserved variables.
- The estimation of parameters.

In the context of bathymetry inversion, data assimilation can be used to benefit from the estimations of multiple numerical models, in order to come up with a single estimated depth value or bathymetry profile having the smallest-possible error.

4 Proposed Method: Deep Learning

In this section, I will present the building blocks that are essential to the work I did during my internship, I will start by giving a brief introduction to the basics of neural networks, including artificial neurons, multi-layer perceptrons, and the mechanism by which a neural network learns. Then I will move on to more advanced topics in computer vision and deep learning. And finally, I will present some of the recent advancements in the field of deep convolutional neural networks, that acted as starting points to the different approaches I tested during my work.

4.1 Artificial Neural Networks

John McCarthy, widely recognized as one of the godfathers of Artificial Intelligence, defined it as “the science and engineering of making intelligent machines.” [21]

Artificial Intelligence is an umbrella term, that includes all types of different study-fields and techniques, that aim at enabling human-made machines to perform tasks that would normally require human intelligence.

Artificial intelligence has many different forms and methods. Artificial Neural Networks are one of those forms, they are algorithms that are inspired by, and modeled after the human brain. They aim at enabling machines to perform tasks, without being explicitly programmed, and they do this by finding numerical patterns in large amounts of data that describe the real-world, be it images, text, sound, or time-series.

4.1.1 Artificial Neuron: Single Linear Decision Function

The basic building blocks of neural networks are artificial neurons, a single neuron, also called a *perceptron* (Figure 2), can be thought of as a single decision function, it

combines inputs $(x_0...x_n)$ from the data with a set of learned coefficients, or weights $(w_0...w_n)$, that either amplify or dampen that input, thereby assigning significance to inputs with regard to the task the algorithm is trying to learn. These input-weight products are summed and then the sum is multiplied by a value called *bias*, which is a learned value that helps the model to fine-tune the importance or impact of a neuron. Finally, the resulting value is passed through a neuron's so-called activation function f (See Section 4.3), or its decision function, that determines the final output of this neuron.

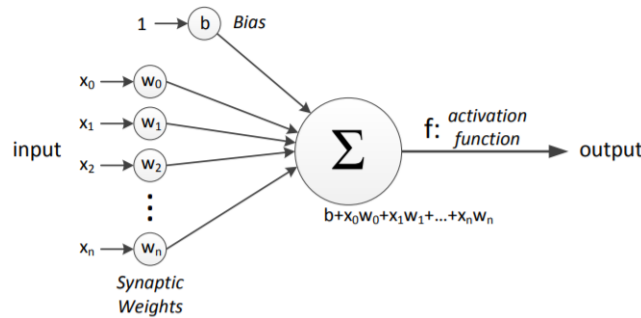


Figure 2: Perceptron Model. Source [12]

4.1.2 Multi-layer Perceptrons

A multilayer perceptron - basic neural network - in its simplest form (Figure 3) is composed of three layers, an input layer, a hidden layer, and an output layer. Each layer is composed of a number of nodes (neurons) where the actual computations take place, as explained previously (Section 4.1.1). Compared to a single neuron, neural networks can be thought of as a set of linear decision functions, that make up a single *non-linear* decision function.

The way these networks work is that each neuron learns its own unique way of splitting the data, and the outputs of all neurons are combined and used to create an overall decision function. In other words, using a network of neurons allows us to create a final decision function that operates on the outputs of other, lower-level decision functions.

4.2 The Learning Mechanism

Learning in all different types of neural networks (including single Perceptron models) is done in three distinct steps, as follows.

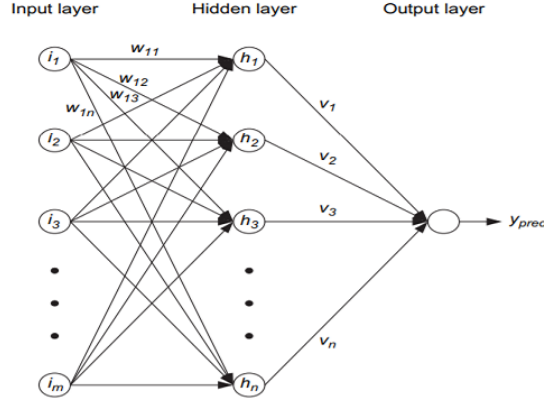


Figure 3: Neural Network with one hidden layer. Source: [2]

4.2.1 Forward-Propagation

First, the network is exposed to the training data by plugging input examples into the input nodes, and passing them through the following layers of the network.

Each node of the network goes through two steps:

1. A node first starts by getting the weighted sum of all of its inputs from previous layers, accounting for the different weight values of each input connection.
2. The weighted sum of the unit's input connections is multiplied by the node's bias, and then passed through its activation function (See Section 4.3).

These two steps are done at all nodes in the network, up until the output node whose value is used as the model's final output or prediction.

4.2.2 Loss Functions

In this step, previously made predictions are compared to the ground-truth values, and the difference between the predictions and the real values is referred to as the *loss* or error value. Loss can be calculated using many different functions depending on the specific task the neural network is being trained to learn. Calculating loss in the case of a binary or multi-class classification problem (categorical values) for example, is different than loss in a regression problem where the predictions are real values (non-categorical).

Having said that, in the case of a regression problem - depth estimation for example, some of the most commonly used loss functions are:

- Mean-Squared-Error (MSE): measures the average of squared differences between the model's predictions and the actual ground-truth values. It is mathematically defined as:

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad (2)$$

- Mean-Squared-Logarithmic-Error (MSLE): A variant of the MSE function, a main difference between the two is that MSLE is based on the *ratio* of difference, instead of the difference itself. This is very useful in cases where small and large differences need to be penalized equally, depending on the task at hand and the type of available data. The mean squared logarithmic error is denoted as:

$$MSLE = \frac{1}{n} \sum_{i=1}^n ((\log(Y_i) + 1) - (\log(\hat{Y}_i) + 1))^2 \quad (3)$$

It should also be noted that depending on the problem at hand, or the goal behind the task the neural network is learning, it is common to create custom loss functions that emphasise a specific desired feature in the data, and which can help the network to learn and reach its objective more reliably.

4.2.3 Back-Propagation & Optimization

After the loss is calculated, it is passed back to the different neurons of the model in a process called *Back-Propagation*, where each neuron receives a fraction of the total loss relative to its contribution to the model's output. This is done so that each neuron can update its own weight and bias parameters, with the aim of giving a better contribution during the next forward pass.

However, there are multiple algorithms that work on actually calculating the different neurons' weight updates based on the previously calculated loss, or in other words, algorithms that aim to optimize the learning of the model. Stochastic Gradient Descent (SGD), and Adaptive Moment Estimation (Adam), are two of the most commonly used optimizers in modern deep learning applications.

Stochastic Gradient Descent

Stochastic Gradient Descent is an iterative optimization algorithm, it aims to minimize the cost or error of a function with respect to its parameters. It uses the first-order derivatives of the objective function with regards to the different weights

involved, in order to calculate each weight's update value. It also makes use of a learning rate parameter η , which determines the size of the update to be done in order to reach a minimum, the value of the learning rate can be modified during training using a pre-defined schedule which allows the model to make significant updates during initial training, and then move to fine-tuning instead of making drastic changes during later stages of the training. The process of calculating the weight updates is done for each data example, or for mini-batches of examples (also called Mini-Batch Gradient Descent).

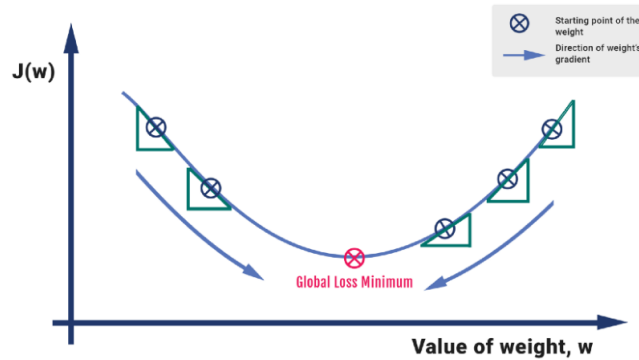


Figure 4: How Gradient Descent works for one parameter (w). Source: [20]

Adaptive Moment Estimation Adam [16] can be seen as a combination of two other optimization algorithms called Adaptive Gradient Algorithm (AdaGrad) [11] and Root Mean Square Propagation (RMSProp) [31]. It maintains a per-parameter learning rate, which is very useful in problems where the gradients can be very sparse such as computer vision applications for example. These per-parameter learning rates are modified dynamically during training based on the values of a parameter's previous gradients, which allows the layers that have been learning very slowly to speed up their learning individually, and gives the model a better chance of not getting stuck in a local minima.

Unlike RMSProp however, in addition to adapting the gradients based on the average first-moment estimates (average), Adam also makes use of the uncentered variance or second-moment estimate (squared average) of the previous gradients. Another important difference between Adam and RMSProp is that Adam uses bias-correction to counteract the initialization bias caused by initializing the moving averages to zero, leading to the averages being biased towards zero.

A weight update using the Adam algorithm is calculated according to the following

formulas (4, 5, 6):

$$v_t = \beta_1 * v_{t-1} - (1 - \beta_1) * g_t \quad (4)$$

$$s_t = \beta_2 * s_{t-1} - (1 - \beta_2) * g_t^2 \quad (5)$$

$$\Delta w_t = -n \frac{v_t}{\sqrt{s_t + \epsilon}} * g_t \quad (6)$$

Formulas 4 and 5 calculate the moving averages described previously, where v_t represents the first-moment estimate, s_t is the second-moment estimate, g_t is the gradient at the current step, β_1 and β_2 are Adam hyper-parameters that control the decay rate of these averages.

And Formula 6 is where the actual weight update is calculated, using the previously calculated values for v_t and s_t , and n as the parameter-specific learning rate.

The following are the three main Adam hyper-parameters that can be modified, and that can have a big affect on how the learning occurs:

- Epsilon ϵ : Fuzz factor to avoid division by zero when the value of the gradient is close to zero. A small epsilon value leads to larger weight updates.
- Beta1 β_1 : controls the exponential decay rate of the first-moment moving average.
- Beta2 β_2 : controls the exponential decay rate of the second-moment estimate.

For a more detailed explanation of how the different optimization algorithms work, please refer to [24].

4.3 ReLU Activation

An important part of neural networks are the activation functions applied to the neurons' outputs, they are functions that decide which neurons should activate (pass their values on through the network). One of the most commonly used activation functions is the Rectified Linear Unit (ReLU) function, which is defined mathematically as $y = \max(0, x)$, meaning that it is an identity function for all positive values, and zero for all negative values. ReLU is very common due to the following reasons:

- It is **computationally inexpensive** therefore it allows for faster model training and running, since it does not involve any complicated mathematical functions.
- It allows for **faster model convergence** compared to other functions. Linearity means that the slope does not plateau or saturate, when x gets large. It does not have the vanishing gradient problem suffered by other activation functions like sigmoid or tanh for example, where the value after activation is usually below 1, making it more likely for the gradient to get smaller and smaller as the network gets deeper.
- It is **sparsely activated**. Since ReLU is zero for all negative inputs, it's likely for any given unit to not activate at all which is often desirable, as it leads to more concise models, that is because it is more likely that neurons are only considering and processing the meaningful parts of the input only, instead of all input including noisy or irrelevant parts. It is also more representative of our actual biological neural networks that ANNs try to imitate, where we have billions of neurons in our bodies, but not all of them are activated at the same time, they have different roles and they fire as a result of different signals. However, this can also lead to issues in the case where a big percentage of neurons in the model becomes inactive.

4.4 Deep Learning & Computer Vision

Deep Learning or Deep Neural Networks are terms for a type of neural network architectures that have been gaining a lot of attention in the last number of years, and which have been used to great success in a number of different fields. In this section, I will explore the different ideas that led to the birth of these deep neural networks. And I will briefly present deep convolutional neural networks which are used for computer vision, since they are designed to work with image or image-like data (satellite imagery in my case).

4.4.1 Computational Complexity

The neural networks we have discussed so far are all considered to be *shallow*, the main difference between these two types is the number of hidden layers they have. Deep networks have 2 or more hidden layers, while shallow networks are composed of a single hidden layer only (Figure 5).

This difference in the number of hidden layers, makes deep neural networks more

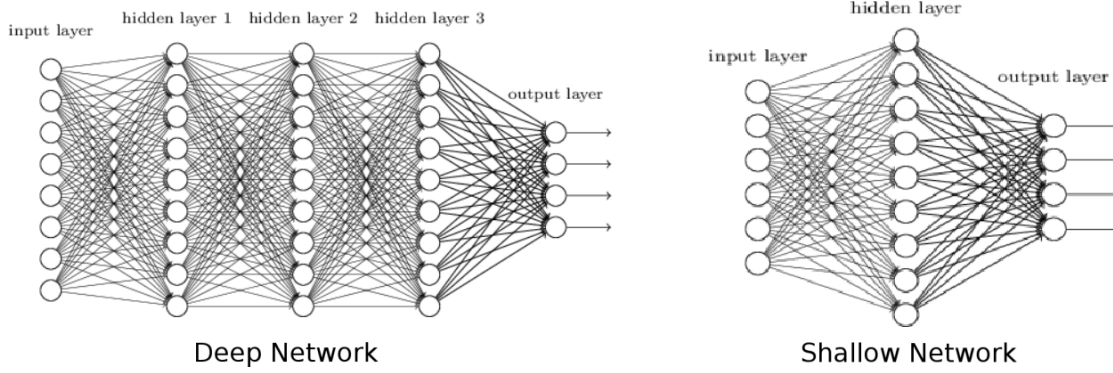


Figure 5: Neural Networks Depth Comparison. Source: [22]

attractive, computationally speaking, than shallow networks having the same number of connections, for tasks that have non-linear characteristics. This is due to the (resulting) difference in the number of connections between the different layers of the model, and therefore the computational complexity of the model. In deep networks, the functions computed by neurons in the lower layers, can be reused by the following layer, and the same is not true in the case of shallow networks. An example of this behavior can be seen in Figure 6 (Red Neurons).

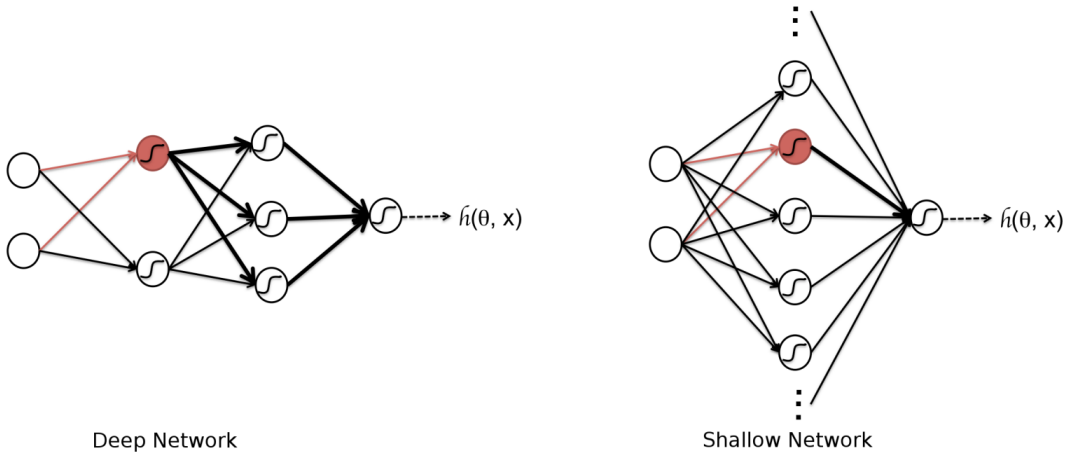


Figure 6: Deep VS Shallow: Neuron Value Reuse [17]

Moreover, all neural networks discussed so far have been *fully-connected* neural networks, where each neuron is connected to all neurons in the previous layer, as can be seen in Figure 5 for example. When dealing with image data however, the number of connections using this model, and consequently the computational complexity, would grow massive very quickly. Considering an image of size 100×100 pixels for

example, each neuron in the first hidden layer only would have 10,000 trainable parameters (weighted connections to the next layer). This computational complexity can be solved by using the following techniques:

- **Local Receptive Field:** where a neuron has a specific receptive field, it is connected to a small number of neurons in the previous layer rather than all (Figure 7), which greatly minimizes the number of trainable parameters.
- **Weight Sharing:** which is an important operation in the field of signal processing, known as *convolution*, where the same filter can be applied to many different positions in the input.

These techniques can be observed in figure 7, where each neuron in the second layer is connected to a subset of the neurons in the previous layer, and where we have the following weight constraints $w1 = w4 = w7$, $w2 = w5 = w8$ and $w3 = w6 = w9$. These constraints allow us to greatly reduce the number of parameters because we no longer need to store all the weights from $w1$ to $w9$, we can instead store the weights $w1$, $w2$, and $w3$ only.

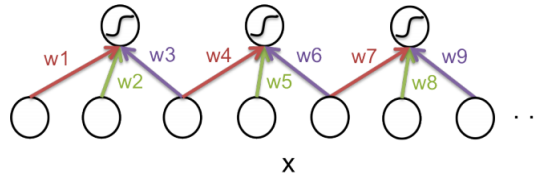


Figure 7: Weight Sharing & Local Receptive Field. Source: [18]

4.4.2 Feature Locality

Feature Locality is another important point that should be considered when dealing with natural data, it refers to the fact that the exact same object or pattern can appear in different positions of the input data, therefore our network needs to be able to detect these objects regardless of their exact positions (the network should be Translationally Invariant). This is done by adding *pooling layers* to the network, also called sub-sampling layers, that aim to minimize a group of values into a single value, the most common of which being max pooling layers. For a more detailed explanation of this type of layers please refer to [18].

4.4.3 3D Filters for Multi-Channel Inputs

When working with image data, we usually have to deal with multiple channel inputs (R,G,B). It is possible to modify the architectures described earlier to work with this type of data, by having filters that each look at multiple channels (Figure 8).

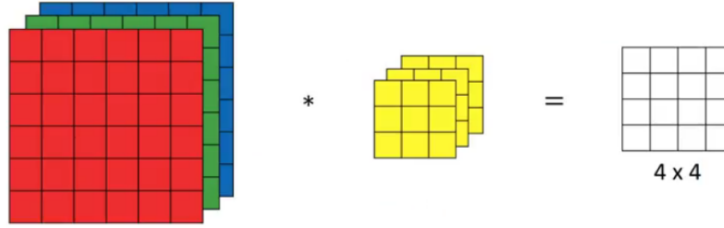


Figure 8: 3-Channel Kernel (middle) applied to an RGB image (left). Source: [27]

4.4.4 Convolutional Neural Networks

From what was gathered and described earlier in this document, we can define a Convolutional Neural Network as a deep learning algorithm, that works with image or image-like input data, assigns importance (learn-able weights and biases) to various features/objects in the input data in order to be able to minimise or maximise an objective function (to perform a task).

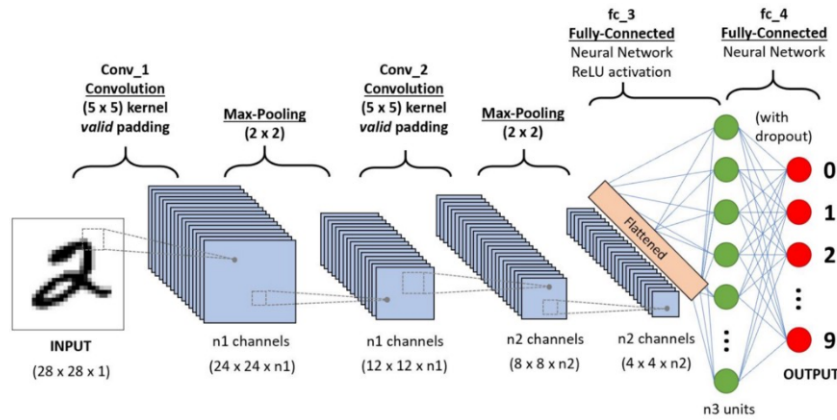


Figure 9: A CNN architecture to classify handwritten digits. Source: [25]

In Figure 9 we can see the overall architecture of a CNN that can classify handwritten digits. We can see that it is composed of three main components:

- Input layer.

- Hidden layers: consisting of couples of convolution + pooling operations, they are responsible for capturing the different patterns and features in the input image, followed by a ReLU activation and a fully connected (network) that works on classifying the features detected in the previous hidden layers.
- Output layer: N output neurons where N is equal to the number of possible classes (digits from 0 to 9 in the case of figure 9).

4.5 Neural Network Architecture Design

In the previous sections, I discussed the architecture and use of neural networks to perform *classification* on non-linear data, using traditional classifier architectures, including Convolutional Neural Networks which are used with image-like input data.

However, one of the main reasons that allows deep neural networks to be so widely used is that they are rather easily adaptable to a large range of different types of tasks. In this section, I will briefly describe some of the important developments that happened during recent years in the field of deep neural networks' architecture design, that led to considerable improvements in the efficiency and performance of these models.

I will also give an introduction to the Encoder-Decoder architecture (also called Auto-Encoders), which is another useful architecture that can be used for tasks other than classification, and which I made use of during the first part of my project (See Section 6).

4.5.1 Very Deep Networks

During 2014, Simonyan and Zisserman found that the depth of the network makes a considerable difference on its accuracy (See table 3, the proposed network is named VGG). One of the possible reasons that enable neural networks to become more robust as they grow deeper, is that they become capable of finding more complicated correlations and relations in the data, allowing them to give better predictions. The importance of this work is that it led to a lot of new ideas, and further improvements in the domain. However, as these models grew in depth, new issues started arising and becoming more apparent such as the *Vanishing Gradients* problem. Where the gradients get so small as they back-propagate through the different layers of the network, that some of the layers end up learning extremely slowly, if at all. This problem was then largely solved by using normalized initialization, and intermediate normalization errors [14].

Method	top-1 val. err. (%)	top-5 val. err. (%)	top-5 test err. (%)
VGG (2 nets, multi-crop & dense eval.)	23.7	6.8	6.8
VGG (1 net, multi-crop & dense eval.)	24.4	7.1	7.0
VGG (ILSVRC submission, 7 nets, dense eval.)	24.7	7.5	7.3
GoogLeNet (Szegedy et al., 2014) (1 net)	-	7.9	
GoogLeNet (Szegedy et al., 2014) (7 nets)	-	6.7	
MSRA (He et al., 2014) (11 nets)	-	-	8.1
MSRA (He et al., 2014) (1 net)	27.9	9.1	9.1
Clarifai (Russakovsky et al., 2014) (multiple nets)	-	-	11.7
Clarifai (Russakovsky et al., 2014) (1 net)	-	-	12.5

Table 3: Comparison with the state of the art in ILSVRC classification (2014) [30]

More details on VGG networks and the work of Simonyan and Zisserman can be found in the article *Very Deep Convolutional Networks For Large Scale Image Recognition* [30].

4.5.2 Residual Neural Networks

He et al. in [14] present the *degradation* problem that occurs during the training of deep neural networks. As the network grows in depth, its accuracy gets saturated and then degrades very quickly, which suggests that not all systems are equally easy to train, and that increasing the depth of a network can sometimes deteriorate its performance and accuracy.

They propose a *deep residual learning framework* as a solution to this problem, where instead of hoping that a set of layers will learn a certain mapping that is embedded in the data, these layers are directed to learn a residual mapping by introducing identity shortcuts (Figure 10) that lead each set of stacked nonlinear layers to learn a residual mapping such as: $F(x) = H(x) - x$ (where $H(x)$ is the desired underlying mapping), and the original mapping is recast into $F(x) + x$. They argue that it is easier to optimize a residual mapping, than to optimize the original unreferenced mapping. Using this method or framework, if an identity mapping was optimal, it would be easier to push the residual to zero, than to learn an identity mapping using a set of stacked nonlinear layers.

Table 4 shows an accuracy comparison based on the 2012 ImageNet image classification dataset [9], between the architecture proposed in [14] that uses identity connections (ResNet), the same architecture without these connections, as well as other well-performing architectures.

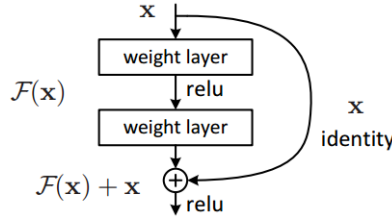


Figure 10: Residual Block. Source [14]

Method	top-1 val. err. (%)	top-5 val. err. (%)
VGG (ILSVRC' 14)	-	8.43*
GoogLeNet (ILSVRC' 14)	-	7.89
VGG (V5)	24.4	7.1
PReLU-net	21.59	5.71
BN-inception	21.99	5.81
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4: Error rates of single-model results on the ImageNet validation set (except for * which is reported on the test set) [14]

4.5.3 The Auto-Encoder Architecture

The idea behind the Auto-Encoder architecture (Figure 11) is to allow a neural network to learn a compressed representation of the knowledge contained in the input data, by imposing a bottle-neck hidden layer where the outputs of all neurons in previous layers in the network are compressed to a smaller number of neurons, before being mapped back to their original shape. In other words, the network tries to learn a mapping function, that can encode data to - and decode it from - a compressed version of itself.

Le et al. give a great explanation of how Auto-Encoders work in [18], using an example of using an encoder-decoder architecture to perform cellphone data usage optimization. Some important points on Auto-Encoders that are worth mentioning:

- Similar to the previously seen classification deep neural networks, Deep Auto-Encoders can be created by adding more hidden layers to the architecture, enabling it to find more complicated relations in the data to be mapped [18].
- Auto-Encoders are commonly used to perform data denoising, by inserting noise into the data to be used as input, and learning to reconstruct the original

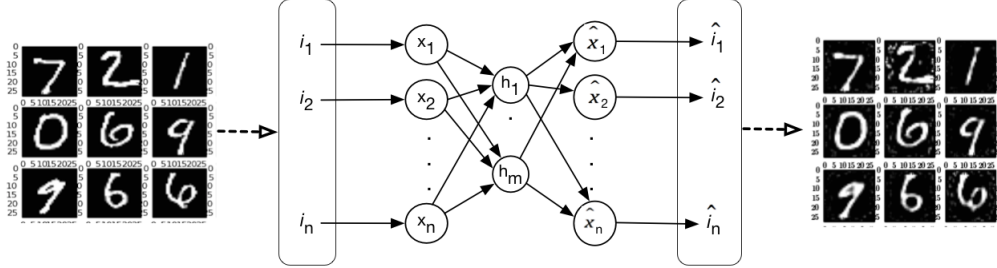


Figure 11: Basic Auto-Encoder Architecture for Image Reconstruction. Source: [13]

noiseless data (Figure 12), in addition to a large variety of other tasks including Object Detection and Localization, Image Segmentation, and Pretraining of Neural Networks, to name a few.

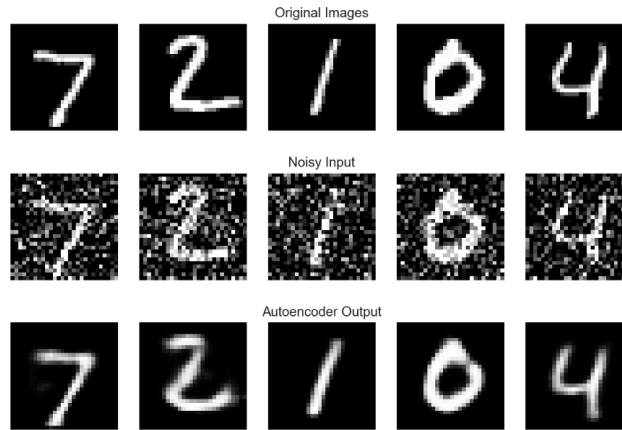


Figure 12: Example Denoising Results. Source: [10]

Neural networks and deep learning are very large fields of research, and they contain many different techniques and architectures that aim to solve a wide range of problems. The aim of this section was to give a brief introduction to the basic building blocks of neural networks, and to present some of the more modern deep learning architectures that were relevant to the work I did during my internship.

5 Synthetic Dataset

Currently, training a deep neural network usually requires having large amounts of example data for the network to learn from. Generally speaking, as the relation the neural network is trying to learn grows in complexity, the more example data given

to the network the better, as allows it to learn a more robust representation of its objective.

Bathymetry inversion is based on finding a physical link between the observable wave patterns and their underlying bathymetry profile, which can be considered as a complex relation. The issue we faced at the beginning of my project was the sparsity of real bathymetric measurements due to the high cost of performing these measurements, which led us to creating a synthetic dataset that imitates the real Sentinel-2 satellite images.

In this brief section, I will start by exploring the different constraints we had to deal with when creating the synthetic data, and how we faced them. Then I will present the different tools we used to create the data, ending with some examples of the data we generated.

5.1 Constraints

Before actually creating our synthetic dataset, we had to consider the following ideas and constraints:

- Minimizing the simulation time: the simulator we used (See Section 5.2) runs its simulations at $1m$ resolution, so the bigger the area simulated the longer it would take to actually run enough simulations to create a dataset.
- Minimizing the training time: The larger the image size, the more time it takes to train per-image.
- Varying wave patterns: physically speaking, the same bathymetry profile can cause different wave patterns, based on wave conditions such as celerity and direction for example.

For all those reasons, we decided to simulate the wave patterns over an area of $2*2km^2$ with $10m$ resolution, giving us $200 * 200px$ images (after downscaling the resolution of the raw simulation data from $1m$ to $10m$). And in order to imitate the different bands from Sentinel-2, we stacked 4 images following the same intervals of the $10m$ bands (See table 2), ending up with a $200 * 200px$ image having 4 different layers.

5.2 Data Creation

The data creation was done using two separate tools, Python and FUNWAVE-TVD. We used Python in order to create random bathymetry profiles, the process of creating a bathymetry profile can be summarized as follows:

- Random slope from a predefined range, such that the max depth (at $2km$ offshore) falls between $40m$ to $100m$ approximately.
- Adding random bumps and sandbars with random height and width parameters.
- Applying smoothing to the edges of the bathymetry, such that the edges on both extremes of the alongshore axis have equal values (a constraint imposed by the wave simulator).
- Validating, and correcting if needed, that no offshore bumps are above sea level.

Due to the wave simulator constraints explained previously, all bathymetry profiles created are initially larger than the area of the profiles that will be used for training later, and they are cropped after the simulation is over (as well as the wave pattern images).

After obtaining a bathymetry grid, it is passed to a wave simulator called FUNWAVE-TVD [28], which is initialized with random wave parameters in order to have varying wave conditions between the different examples. Once the simulation is done, Python is used again to format the data (cropping the margins as explained previously), normalize it, and extract separate image bursts as *numpy* arrays so they can be used during training.

5.3 The Processed Data

Finally, this subsection aims to present some examples of the data, in order to get a better understanding of its form and structure.

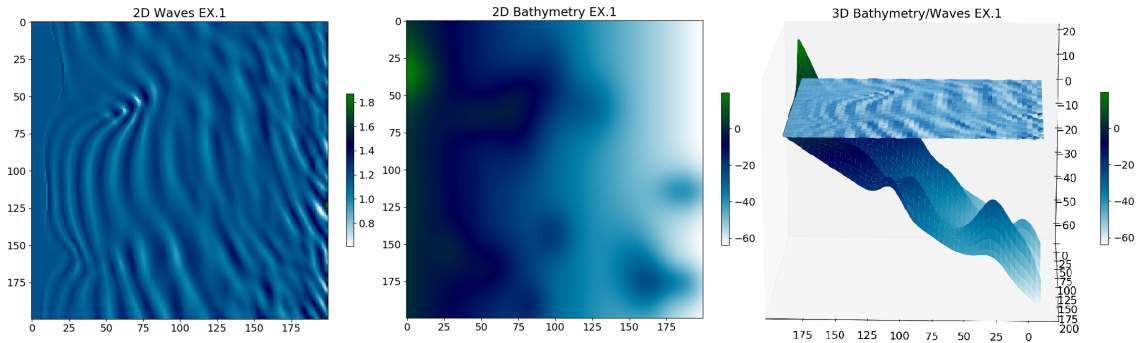


Figure 13: Data Example 1 (Single wave frame)

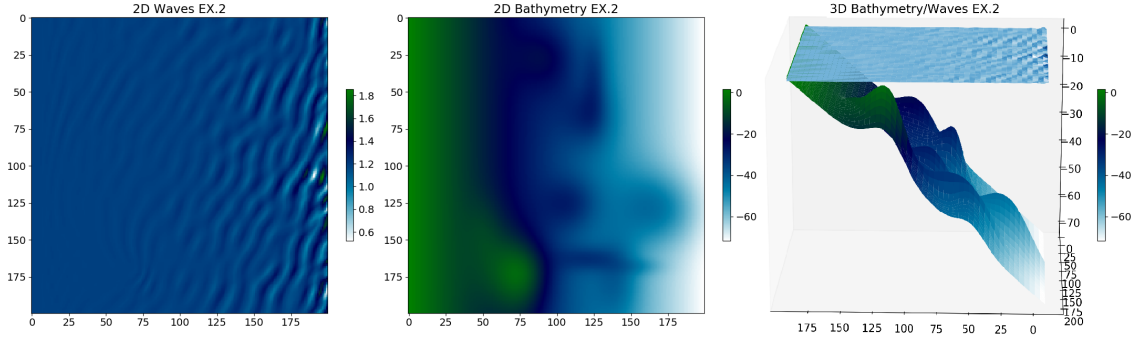


Figure 14: Data Example 2 (Single wave frame)

Figures 13 and 14 show two different example bathymetry profiles, along with a corresponding wave pattern for each. These wave patterns shown are a single frame out of the 4 frames (layers) used in a training image burst. From the figures, we can see the difference in wave conditions between different example patterns, and how each wave pattern develops according to its underlying bathymetry.

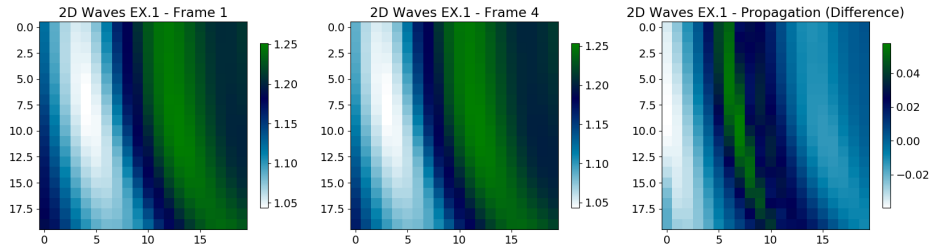


Figure 15: Data Example 1 - Sub-window Wave Patterns (Left and Middle) - Wave Propagation Between Frames 1 and 4 (Right)

Figure 15 shows the wave propagation, over a sub-window taken from the middle of the full area shown in 14, between the first and last frames of that image burst. Since the whole burst consists of 4 frames over one second (See table 2), we can see that the wave propagation is not easily realised visually. However, the difference between the first and last frames (Figure 15 - Right) shows the amount of wave movement over one second.

Using these tools, we were able to create two different datasets, one for each deep learning approach tested, that were good enough as a starting point for this research

project. However, this step of the project should be developed and improved further in order to contain more realistic features (Further exploration, see section 8).

6 Large-Scale 2D Inversion

For my first approach, we wanted to train a neural network to reconstruct complete $2 * 2km^2$ bathymetry profiles based on the wave patterns observable in the corresponding full image bursts of the same size ($2 * 2km^2$). To do this, I relied on the Auto-Encoder architecture, using the full-sized $200 * 200 * 4px$ image bursts as input data.

As discussed previously in section 4.5.3, Auto-Encoders work very well when trying to map inputs to outputs of the same dimensions, by finding hidden relations between patterns in the input and the output data. The pattern in this case is the evolution of the wave pattern over time, that can be observed over the different layers of each image, since each layer represents the sea state at a specific point in time. And the relation the Auto-Encoder would try to find, is the actual physical relation between the wave patterns and the underlying bathymetry profile.

6.1 Base Architecture

The specific Auto-Encoder architecture I used as a starting point is called U-Net, it was proposed in 2015 by Ronneberger et al. in [23], and since then it has gained a lot of popularity, and has been used for multiple types of applications including pixel-wise regression [33], and image segmentation [7, 15].

U-Nets build upon Fully Convolutional Networks (FCN) that were proposed by Long et al. [19]. These architectures are aimed at performing image segmentation tasks, where the output of the network has the same x, y dimensions of the input data, and where each pixel in the output is classified into a specific class. FCNs do this by either directly up-sampling the output of different convolutional layers in the network (FCN-32s), or by combining higher resolution features from the contracting path with the up-sampled output (FCN-16s, FCN-8s), as can be seen in figure 16.

On the other hand, U-Nets expand this idea further, they improve performance and accuracy by conserving and propagating useful features through the up-sampling portion of the architecture, which is done through the use of large feature maps in the up-sampling layers in addition to the (traditional) approach of having large feature maps in the down-sampling layers. This results in the symmetry between

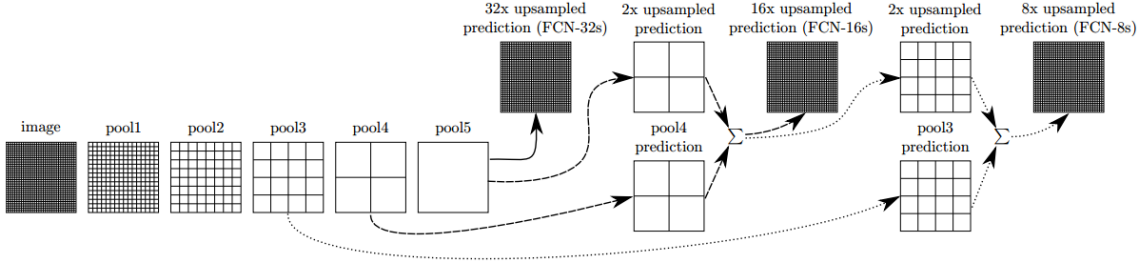


Figure 16: Fully Convolutional Network as proposed by Long et al.. Source: [19]

the down-sampling and up-sampling sections of the network, producing a u-shaped architecture hence the name "U-Net".

6.2 Technical Specification

In this section, I will present the details of the dataset I generated for this approach, and then I will go through the different modifications I made to the original network architecture (U-Net) in order to conform with the format of my data.

The Dataset

For this approach, I used the tools and methods described in section 5 in order to create 1.5k different random bathymetry profiles, and for each profile I ran 16 different simulations with random wave parameters. Finally, I extracted 50 wave bursts from each of those wave simulations, ending up with around 1.2 million different $200 * 200 * 4px$ images, resembling the 4 Sentinel-2 bands having 10 meters resolution (See section 3.1, table 2) and covering an area of $2 * 2km^2$.

The Architecture

The original architecture (U-Net) was designed to work with single layered images of size $572 * 572px$. Therefore, the architecture had to be modified slightly in order to be usable with our dataset since it is composed of $200 * 200 * 4px$ images. In the interest of completeness, these changes can be summarized as follows:

- Decreased Input dimensions.
- Applying $4*4px$ padding to the input, because the output dimensions would be less than $200 * 200px$ otherwise, due to the number of times the dimensions of the data are halved between the different down-sampling layers of the network. This also made it necessary to apply cropping to the output of the network, in order to have the desired output dimensions.

- Crop-and-copy connections were replaced with copy-only connections, since the dimensions of the data being concatenated are equal.

The final architecture I used for this part of my project can be seen in figure 17.

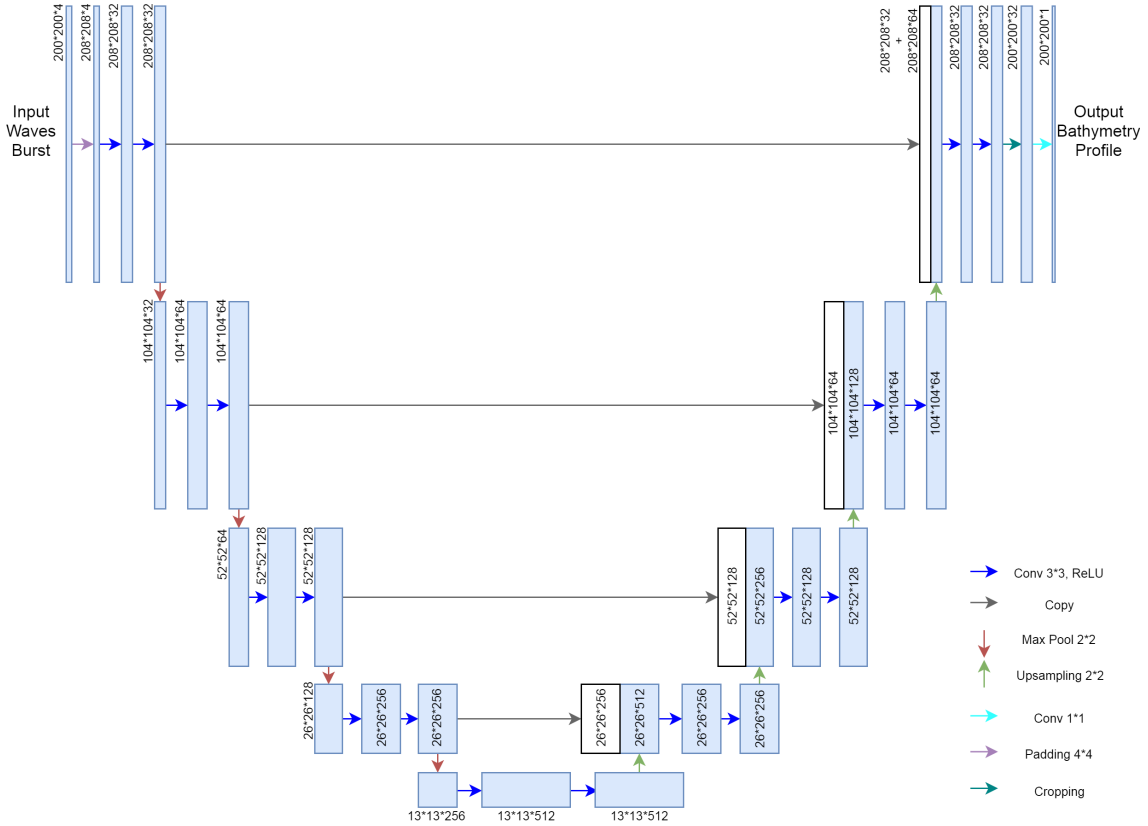


Figure 17: Modified U-Net Architecture.

6.3 Experiments

During this first approach, I relied mainly on SGD as the learning optimizer, and my experiments were focused on comparing the effects of using different loss functions and bathymetry normalization schemes.

6.3.1 Loss functions

In addition to MSE and MSLE, which were described previously in section 4.2.2, I defined a custom loss function that uses the maximum error over the full batch

of training examples as the error value for gradient descent, which I refer to as MAXERROR. The idea behind this loss function was to guide the training to focus on detecting individual features and bumps in the bathymetry, as opposed to considering an average error over the complete bathymetry grid, which could lead to overlooking features limited to a small area of the total grid.

Loss Function	Bathymetry Normalization	rmse	avg. Slope error
mse	10	21.46	0.33
msle	10	8.96	0.11
maxerror	10	5.95	0.05

Table 5: Initial SGD models trained using 100k examples.

From table 5, we can see that MSE was the least performant of the three loss functions tested, therefore much of the further testing will be done using MSLE and MAXERROR only.

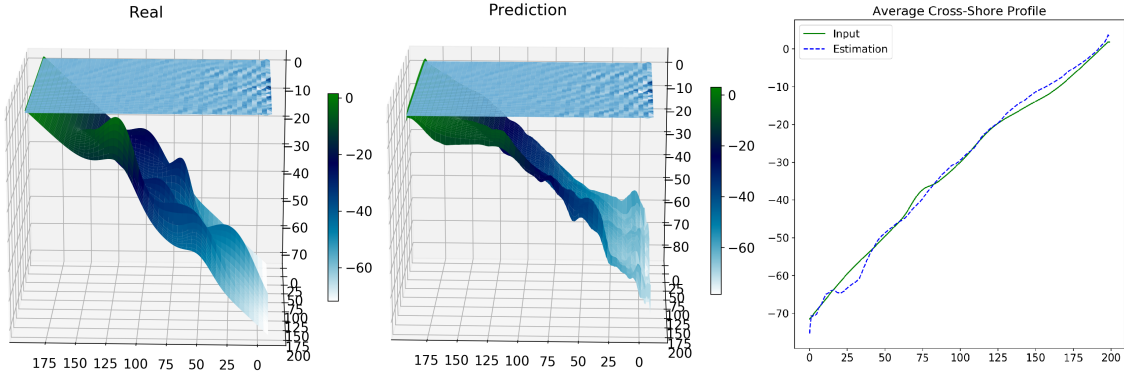


Figure 18: Initial MSLE Result visualization. Model params: {SGD, MSLE, 100k examples, bathynorm 10}

From the example results shown in figures 18 and 19, we can observe how the MSLE model goes for a smoother slope and surface, since it is trained using an overall evaluation of the profile, which allows it get an accurate average profile slope estimation (shallow water is more accurate, this will be discussed further in section 8). As opposed to the model trained with MAXERROR as a loss function, which is able to detect the individual bumps more reliably, however this can lead to a very bumpy surface which is not as realistic as that given by MSLE.

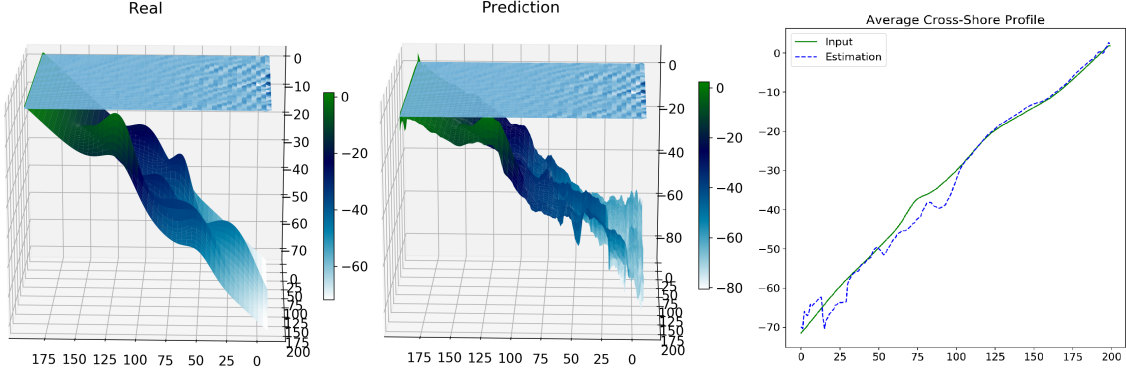


Figure 19: Initial MAXERROR Result visualization. Model params: {SGD, MAX-ERROR, 100k examples, bathynorm 10}

6.3.2 Depth normalization

The second parameter we experimented with was the depth normalization scheme. Since the depth values can go as low as 100 meters, which is a very large value compared to the values of gradients - and consequently the weights -, as well as the wave bursts' values which are always normalized to range between 0 and 1 (to mimic the real satellite data), it was important to normalize the depth values to fall in a range of smaller values, in order to make it easier for the model to actually learn to predict the depth. We tested normalizing by 100 and by 10, having 1 and 10 as the normalized maximum depth values respectively (the maximum real depth value in the dataset is 100 meters).

Loss Function	Bathymetry Normalization	Best Epoch	rmse	avg. Slope error
msle	10	97	8.96	0.11
msle	100	98	13.14	0.13

Table 6: Bathymetry Normalization Comparison.

From table 6, we can see at a glance that using 10 instead of 100 as the depth normalization parameter would be better overall. When looking deeper into the results (see figure 20 for example), we can see that even though the target average profile and the profile produced by the 'norm100' model have very similar shapes overall, the model's prediction was clearly off by 5 to 10 meters in all points. This almost equal distribution of error over the cross shore profile suggests that the model was able to capture and predict the general shape of the bathymetry or its slope from the input bursts. However, the error could be caused by the very small differences

(numerically) between the *normalized* depth values, making the model unable to learn and capitalize on those differences.

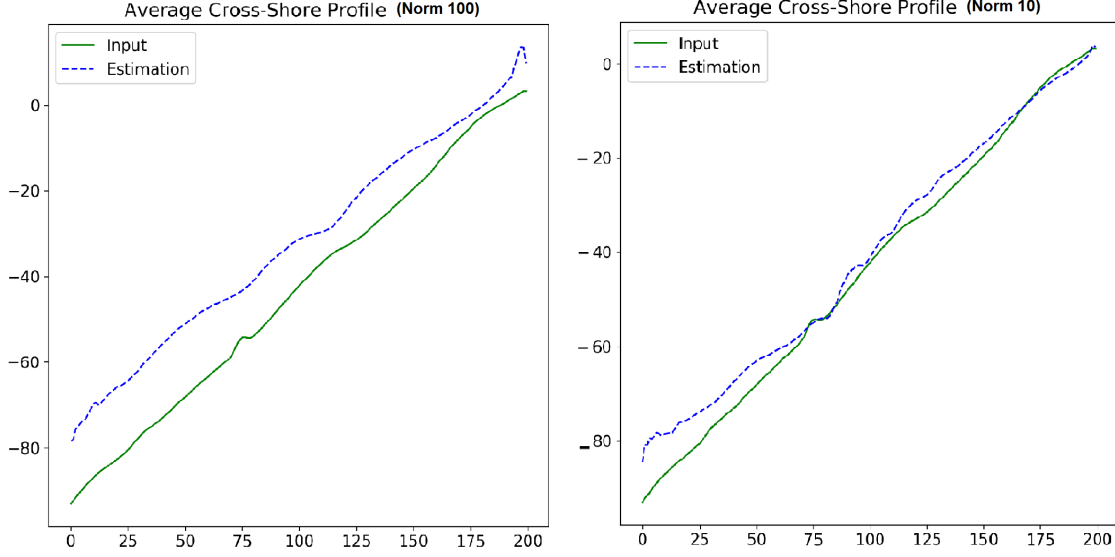


Figure 20: Bathymetry Normalization Comparison. Model params: (SGD, MSLE, 100k examples, bathynorm [left: 100, right: 10])

For those reasons, we can conclude that when scaling the bathymetry by 100, a lot of details can be minimized greatly and therefore lost during training and in the prediction. Which is why all further experiments were done by normalizing the depth values by 10 only.

6.4 Results Analysis & Conclusion

Going further with this approach, we trained two models on the full dataset, using both MAXERROR and MSLE as the loss functions, table 7 summarizes the results we obtained.

Loss Function	Bathymetry Normalization	rmse	avg. Slope error
msle	10	5.87	0.07
maxerror	10	11.22	0.1

Table 7: Bathymetry Normalization Comparison.

During this approach, and out of the different methods tested, MSLE was the best performing and most reliable overall. MAXERROR had an interesting degradation of performance when trained on more data, this can be seen when comparing the results obtained from the models trained with the full dataset in table 7, to the results produced by the initial MAXERROR model trained on 100k examples as can be seen in table 5, which - interestingly - had the lowest RMSE and slope errors out of all the models we tested.

The only major issue we encountered during this part of the project was related to the way the data was generated. During the creation of the data, we overlooked the problem that at the point where FUNWAVE (see section 5.2) creates the synthetic waves (offshore), the waves always look extremely random, which is normal from a technical point of view. However, and considering that we did not account for enough margins in the complete raw simulated area, this meant that our final input image bursts always had very random offshore wave patterns, which greatly affected the models' ability to estimate the underlying depth at those points (See figures 21, 22). This issue was later resolved when creating a dataset for the second approach (See section 7).

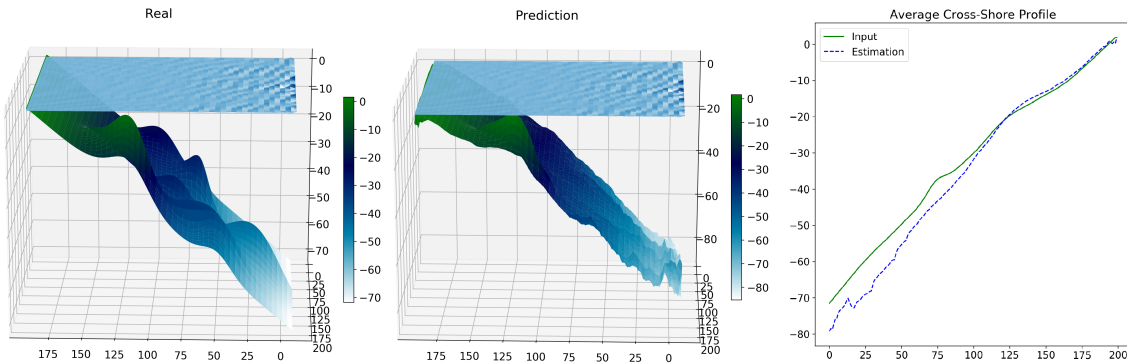


Figure 21: Example 1: Final estimation showing the random offshore wave patterns problem. Model params: {SGD, MSLE, full dataset, bathynorm 10}

Moreover, when comparing our synthetic input data to the real satellite data, we can see how the synthetic data is too perfect in that it always has a straight shoreline at the exact same point, which is not the case in real-life shorelines (See figure 23). In addition, the real data could sometimes have obstacles and foreign objects such as boats or wave breakers for example, or it could be completely off-shore with no shoreline at all, and these types of differences would definitely cause an issue to a model trained with (static) features like our first synthetic dataset.

Another interesting point that came up during this approach, was whether it was actually possible to accurately estimate the underlying depth at deep waters. Phys-

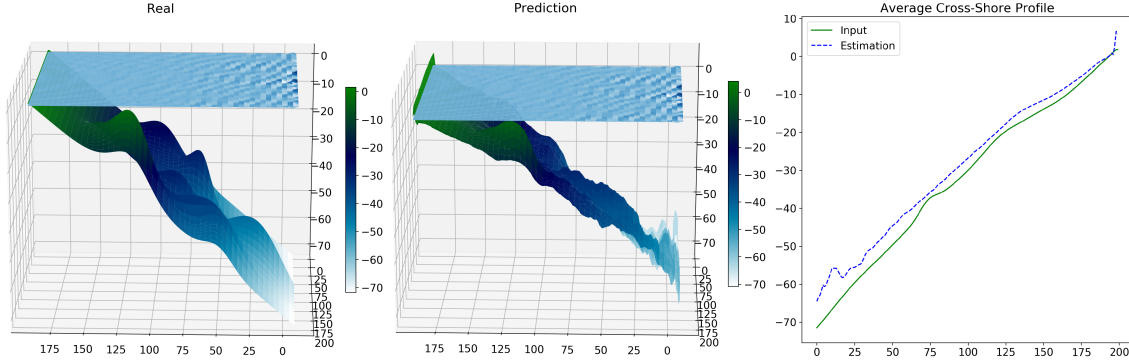


Figure 22: Example 2: Final estimation showing the random offshore wave patterns problem. Model params: {SGD, MSLE, 500k examples, bathynorm 10}

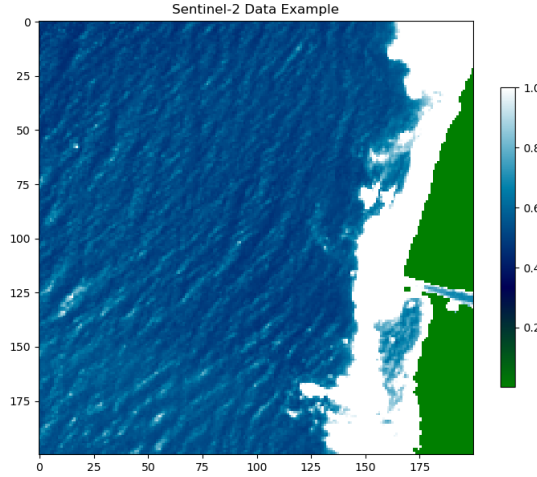


Figure 23: Example Sentinel-2 Image.

ically speaking, the deeper the water is, the more it is true that waves patterns are based on factors other than the underlying bathymetry, such as wind direction and speed for example. However, the exact depth where this becomes true is still an open research question (Further exploration possibility, see section 8). Meanwhile, we can see from the average profile shown in figure 18 for example, that our deep learning model was able to accurately estimate the depth down to 70m, which creates a question of whether this accurate prediction was actually reliable, or if the first method was too biased towards some features in the synthetic dataset (relation to the shoreline for example) that helped the model to make an accurate estimation at that depth, such features which might not be available in the real cases as described previously.

This first part of the project was very useful because it gave us a deeper understanding of the constraints that we needed to respect, in order for the model to be applicable to the real data. However, for all the reasons explained above, we decided to stop this approach and move on to a completely new method that should be more usable in a real use-case.

7 Small-Scale Single-Point Inversion

For my second approach, we wanted to try a more sub-window focused approach, in order to be sure that model is not making use of any patterns in the synthetic data that might not be available in the real data, such as the availability of a shoreline at the same position (if at all) for example.

From a physics point-of-view, a satellite image burst (using the 10m resolution bands) covering an area of $100 * 100m^2$ contains enough information to estimate the average depth beneath that area, with an error margin of 1 to 6 meters. Therefore, theoretically speaking, using the same type of data, a deep learning model should be able to find that physical relation between the observed wave patterns and their average underlying bathymetry.

7.1 Base Architecture

The network architecture I used for this approach is called ResNet, it was proposed in 2015 by He et al. in [14]. ResNets are inspired by the work of Simonyan and Zisserman in [30], where they empirically show that deeper networks are able to learn more complicated relations, enabling them to improve their performance and accuracy. They propose VGG nets that are composed of 16 or more convolutional layers, which at the time were considered very deep in comparison to other state-of-the-art architectures of the time (2014).

He et al. build upon the ideas in [30] by experimenting with even deeper plain networks (i.e. without identity shortcuts). They found that very deep plain networks suffered from the *degradation* problem, and proposed a *deep residual learning framework* using identity shortcuts to solve this problem (See section 4.5.2).

The deep residual learning framework proposed in [14] is used to create "network-in-network architectures" (Figure 24), where a neural network can be seen as a sequence of *residual blocks*, as a result of adding these identity connections.

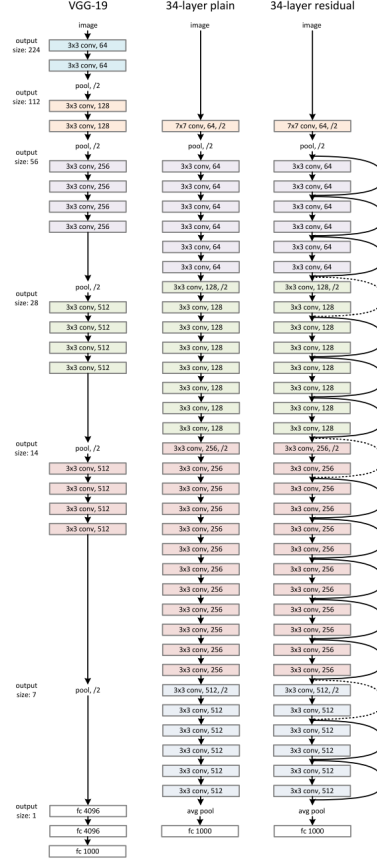


Figure 24: VGG-19 (left), plain (center), ResNet34 (right). Source: [14]

In their paper, the authors experiment with varying depth residual networks, the varied depths impact the number of residual blocks and/or the number of layers within the residual blocks themselves. For a more in-depth explanation of ResNets, please refer to the original work in [14].

7.2 Technical Specification

The aim of this section is to describe the dataset I worked with for this approach, and to document the changes that I made to the original architecture chosen (ResNet) in order to be usable with my specific data.

The Dataset

For this approach, we used the same tools and methods described in section 5 in order to create our synthetic dataset. The following modifications to the data were

done, to get our final dataset:

- Added pits with random features when generating bathymetries.
- Extracted random $20 * 20px$ images ($200 * 200m^2$) from the full-sized data, to be used as input data.
- Extracted $20 * 20px$ from the bathymetry grid that corresponds to the image extracted, and using the average depth value as the target value to be predicted.
- All data extracted is off-shore (i.e. after the wave breaking region), since wave breaking is not currently taken into account during the simulation (see section 8).

Our final dataset was composed of more than $300k$ images of size $20 * 20 * 4px$, along with their corresponding average bathymetry values. In addition to $1.6k$ examples to be used as a validation set that is never included in training.

The Architecture

Since the original ResNet architecture was designed to classify $224 * 224px$ images, the following modifications had to be done:

- Modified input/output dimensions to take $20 * 20 * 4px$ images as input, and to output a single value (average depth).
- Since the problem at hand is a regression problem, I replaced the final Softmax activation with ReLU, in order to output a single value estimation instead of class probabilities.

In order to create the models, I used the default Keras [8] implementation of ResNet, which allows to build multiple ResNet models with different depths. For my work, I mainly used ResNet56 and ResNet110, having 56 and 110 layers respectively.

7.3 Experiments

In this section, I will start by explaining the initial tests I did, and analyzing the results I got, and then I will follow through with the succession of experiments and analysis, ending with the current state of the project at the end of my internship.

An important point to note, is the way I evaluated the different models during this approach. The three following measures were used as a basis for evaluation:

- Root-mean-squared-error (RMSE): Measures the average error in meters. An rmse of 10 for example, means that the model's predictions were off by 10 meters on average, compared to the ground-truth values in the validation set.
- Max-error: Shows the maximum error the model had when tested on the validation set.
- Standard-deviation (STD): Gives an idea of how well the model is able to generalize its depth estimation based on the different input wave patterns. A higher std value suggests that the model is able to find usable information in the input data, on which it can base its predictions.

7.3.1 Initial Models

At the beginning, I started by training multiple ResNet56 models on $25k$ examples, using MSE as the loss function, with varying durations of training (number of epochs), and optimizers (SGD, Adam) with default hyper-parameters.

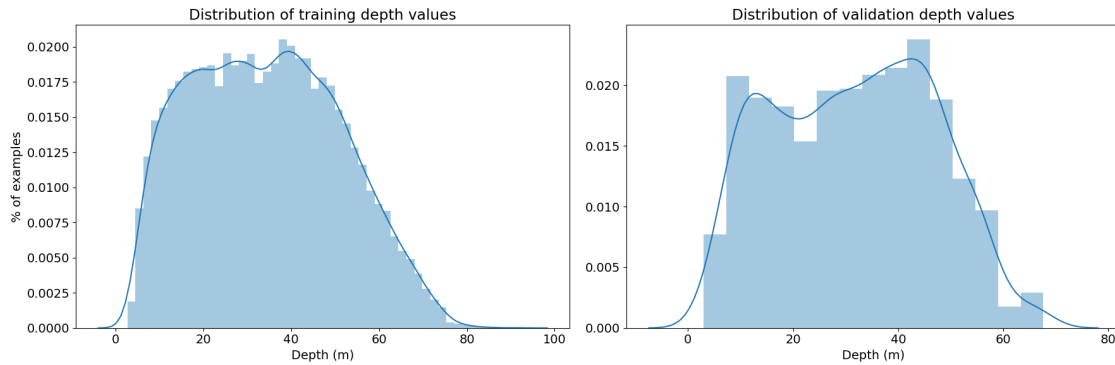


Figure 25: Distributions of the initial training ($25k$ examples) & validation ($1.6k$ examples) datasets as points of reference.

Figure 25 shows the distributions of target depth values in the training ($25k$ examples) and validation ($1.6k$ examples) sets, having average values of 34.28 meters, 31.62 meters, and standard deviations of 16.8 and 15.17, respectively.

My initial models using SGD and Adam with default parameters show a similar behavior of estimating depth values that are very close to the average value in the training set, producing a very concentrated distribution of predictions ($std = 4$), which is not the case in the real data. This behavior can be seen in figures 26, 27.

Moving further with my experiments, I tested training a deeper model (ResNet110) using the same training set ($25k$) and the same default hyper-parameters, for 200

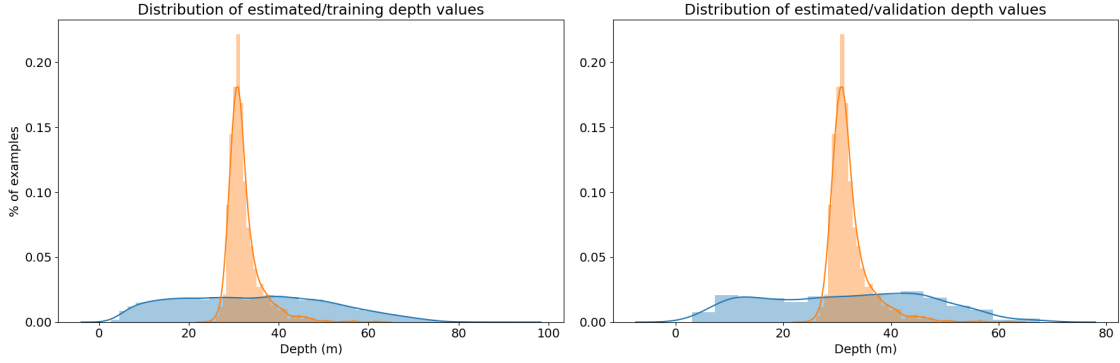


Figure 26: Initial ResNet56-SGD model results distribution: concentrated average estimation problem. Model params {SGD, 25k examples, 100 epochs}, $std = 3.86$, $rmse = 14.76$

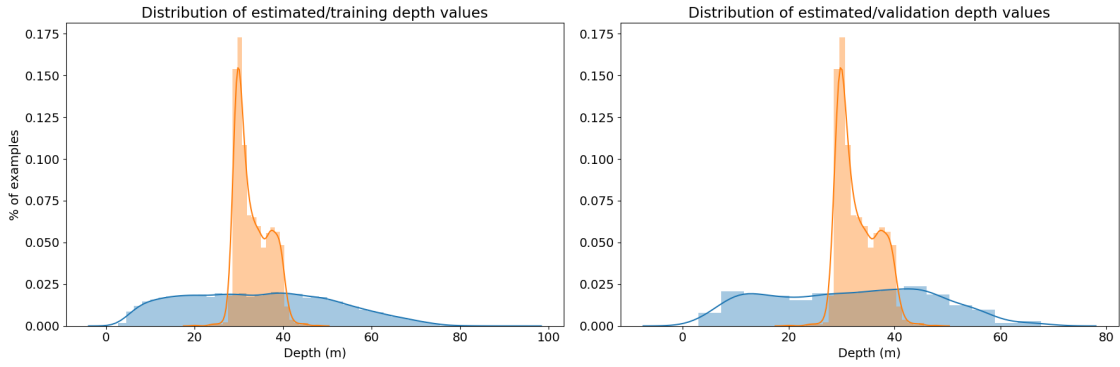


Figure 27: Initial ResNet56-Adam model results distribution: concentrated average estimation problem. Model params {Adam, 25k examples, 100 epochs}, $std = 3.73$, $rmse = 14.83$

epochs instead of 100 only. The results I obtained (Figure 28) suggest that the model is not just simply learning the average value in the training set, but that the model is actually able to find useful information in the input data to learn from, since the standard deviation of the predictions is much higher than the initial models (figures 26, 27). However, the value of root-mean-squared-error was still relatively high ($12.34m$).

The results of this first phase of experimentation showed that the models are able to find useful information, but are not yet able to be trained efficiently to make accurate predictions. Which led to the experiments in the following sections, where different combinations of hyper-parameters are explored in an attempt to improve the learning process.

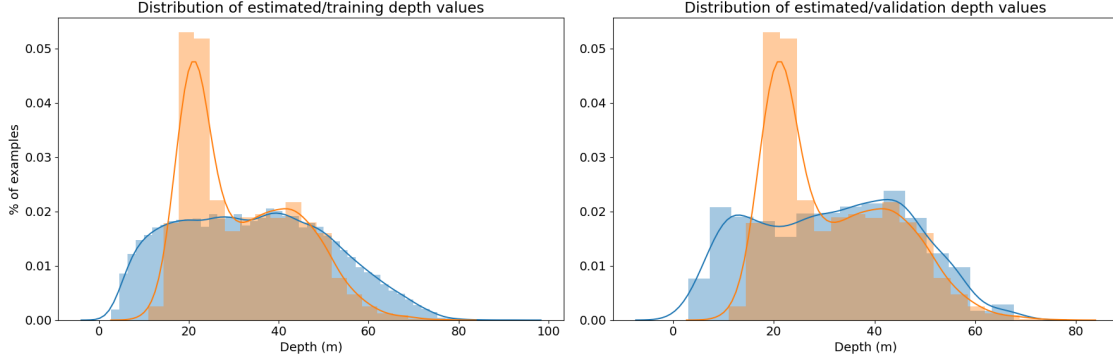


Figure 28: ResNet110-Adam model results distribution: improved STD and RMSE of the predictions after increasing the number of training epochs. Model params {Adam, 25k examples, 200 epochs}, $std = 12.16$, $rmse = 12.34m$

7.3.2 Hyper-Parameter Grid-Search

Based on the previous results obtained, this experiment was focused on exploring different Adam hyper-parameters in order to optimize the learning process.

As explained earlier in section 4.2.3, Adam has three main hyper-parameters (ϵ , β_1 , β_2) that can be modified and that can have a big impact on the learning behavior. In order to explore and find more efficient combinations of parameters for my specific case (i.e. non-default values), we performed a grid-search experiment using the following ranges of hyper-parameter values:

- ϵ : {1, 1e-1, 1e-2, 1e-4, 1e-6, 1e-8}
- β_1 : {0.1, 0.5, 0.9, 0.99, 0.999}
- β_2 : {0.999, 0.1, 0.9, 0.99, 0.99999}

We trained a total of 150 models using ResNet56 for 50 epochs on a dataset of 25k examples, using different hyper-parameter values from the ranges listed above, and we compared the effect of changing each parameter on the final results of the model ($rmse$).

The results of this experiment, including the samples shown in 29 and 30, can be used as a starting point for two other possible experiments. The first possible experiment, which will be explored in the future (Further exploration, see section 8), is based on the patterns we found where using a smaller epsilon (ϵ) value (less or equal to $1e-04$) usually led to a better learning behavior, with β_1 and β_2 falling between 0.9 and 0.99. This information can be used to conduct a more in-depth hyper-parameter search

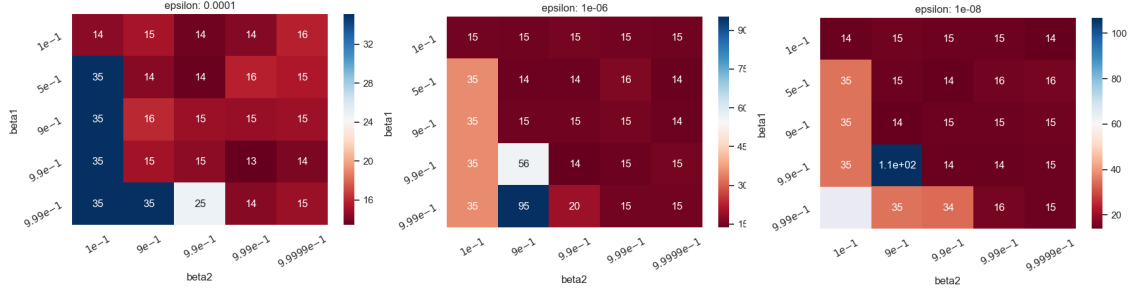


Figure 29: Sample Grid Search Results - Epsilons.

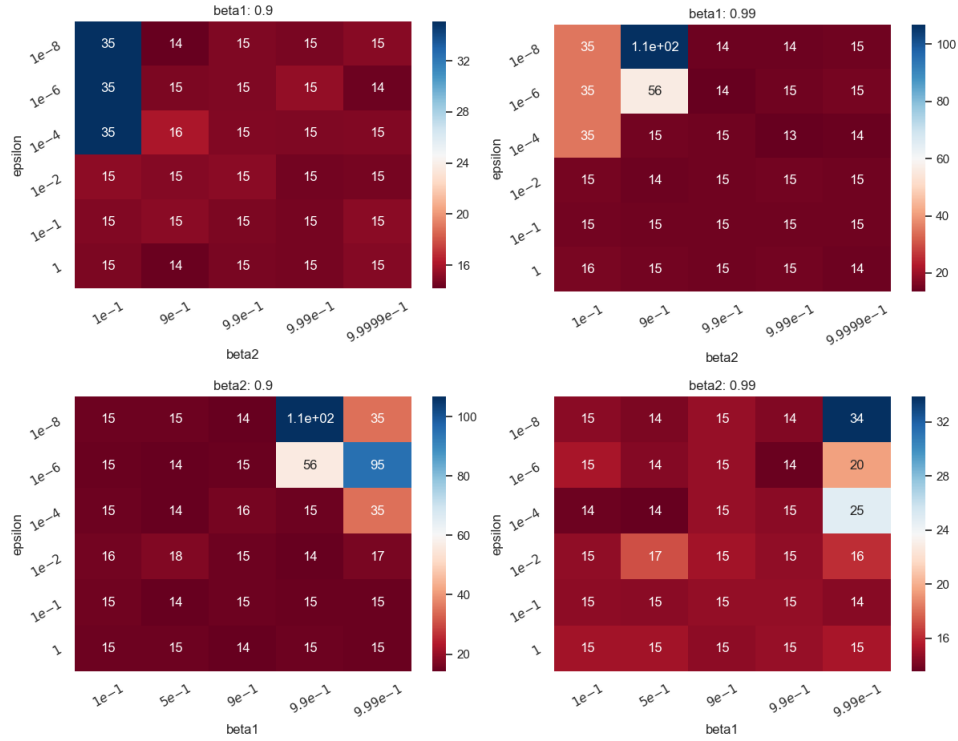


Figure 30: Sample Grid Search Results - Betas.

experiment using a larger set of values falling within the ranges we found to work best. The second possible experiment, and which I had the chance to work on during my internship, was to evaluate the 150 trained models obtained from the previous experiment in the same manner as was done with the previous models (Section 7.3.1), in order to pick a small number of the best performing hyper-parameter combinations, to be used in larger training experiments (more data, deeper models).

Table 8 shows the resulting models and hyper-parameters that we decided to con-

epsilon	beta1	beta2	rmse	max_error	std
1e-04	0.99	0.999	13.43	53.93	8.2
1e-04	0.5	0.99	13.56	55.61	13.45
1e-06	0.999	0.999	15.14	67.94	16.6

Table 8: Grid-search experiment results: Best performing hyper-parameter combinations.

tinue working on. These combinations were chosen as the most interesting to keep working on for the following reasons:

- 1st set {1e-04, 0.99, 0.999}: The model trained using this hyper-parameter set had the lowest rmse and max_error values of all the models trained.
- 2nd set {1e-04, 0.5, 0.99}: The second model had relatively low rmse and max_error compared to the other models, with a high std value compared to the first model, making it an interesting candidate to test.
- 3rd set {1e-06, 0.999, 0.999}: Even though this combination had high rmse and max_error values, having the highest std value of all models trained made it an interesting combination to explore further. This is because the higher the std value, the more likely that the model is not just producing (average-value) estimations.

7.4 Results Analysis & Current State

The hyper-parameters found from the previous experiment were used as a base to create a small number of different combinations, to be used to train multiple models on 50k examples, using deeper architectures, for 200 epochs.

model	epsilon	beta1	beta2	rmse	max_error	std
ResNet110	1e-06	0.5	0.99	12.87	40.14	6.46
ResNet56	1e-04	0.99	0.999	13.19	58.53	8.26
ResNet56	1e-04	0.999	0.999	12.98	56.2	9.83
ResNet110	1e-04	0.99	0.999	14.49	39.23	5.48

Table 9: Larger training experiments (50k examples, 200 epochs) using the combinations found from the grid-search results.

Table 9 shows some of the final models I trained with a brief evaluation of each. We can see that on average, most models were able to improve their performance slightly compared to previous results (table 8).

As my final experiment during this internship, I trained one last model using the best performing parameter set from table 9 ($\epsilon = 1e - 06, \beta_1 = 0.5, \beta_2 = 0.99$), with an even larger dataset than before (100k examples), in order to see whether the model would improve or not.

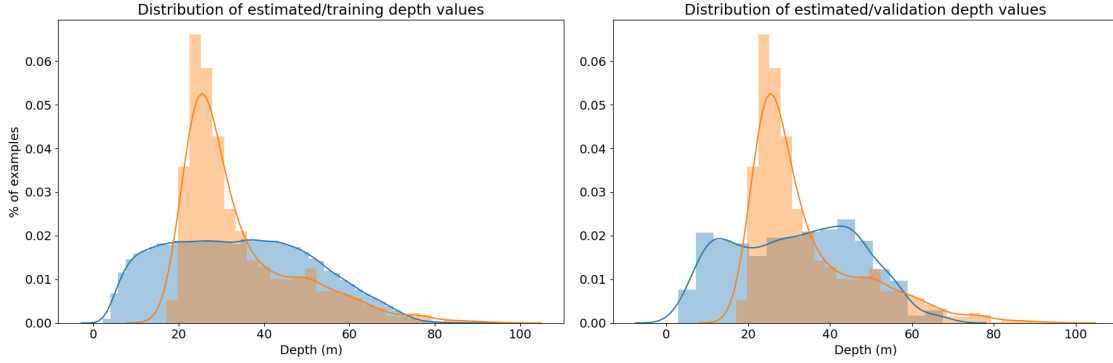


Figure 31: ResNet110 model results distribution. model params {Adam, 100k examples, 200 epochs}, $std = 13.57$, $rmse = 11.63m$, $max_error = 47.53m$

As can be seen in figure 31, and comparing to the previous result obtained with the same parameters (first row in table 9, values: $rmse = 12.87$, $max_error = 40.14$, $std = 6.46$), doubling the dataset size enabled the model to learn a better representation and improve its overall accuracy.

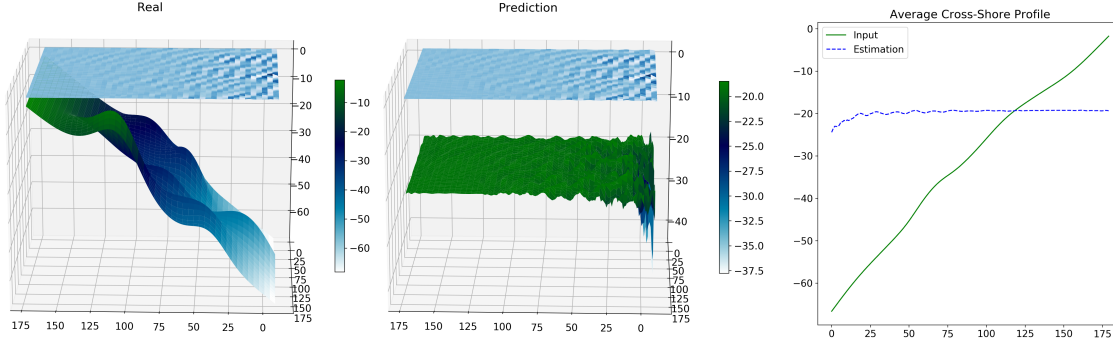


Figure 32: Reconstructed Bathymetry using the final ResNet110 model. model params {Adam, 100k examples, 200 epochs}, $rmse \approx 20m$

Finally, using this final model, I used a sliding window technique to reconstruct a complete bathymetry grid from a full-sized wave image burst. The reconstructed bathymetry can be seen in figure 32.

After performing a comprehensive hyper-parameter search using modern deep learning architectures (ResNet56, ResNet110) and a large set of examples that contains

a wide variety of depths and wave conditions, this approach still remains an open question. However, knowing that this problem is - theoretically - solvable, the final results I obtained only prove that bathymetry inversion is a difficult problem for the current state-of-the-art in deep learning, which should only act as a driver for further deep learning research and development.

8 Discussion & Conclusion

Deep learning is an on-going field of research that keeps on improving as further research is conducted and newer ideas are explored. The same goes for my research project during this internship, where even though I was able to apply two different deep learning approaches to the problem of bathymetry inversion, there are a lot of different ideas still left to explore, in addition to exploring the two current methods even further. At the end of my internship, I am left with a number of ideas, and ways for improvement, that I would have liked to try but was not able to due to time constraints. However, these ideas will (hopefully) be explored and tested in the near future!

First, the data creation part of the project is very essential when wanting to actually apply the trained models to real data. This part (data creation) can be split into three different steps:

Bathymetry generation: Due to the wide variety of seafloor environments (sandy, muddy, rocky... etc.), and the specific features of each type which affect how the corresponding bathymetry develops and reacts to the natural forces surrounding it, the task of modelling and creating synthetic bathymetry profiles that actually resemble real cases can get so complex and detailed, that it could be considered its own field of study.

However some of the ideas that can be - easily - incorporated, and that can help introduce more natural and realistic features into the synthetic dataset are as follows:

- **Controlled randomness:** Which refers to having a more physics-based approach to creating the random bumps and pits in the bathymetry, in order to have more realistic data for training. In addition to having both types of offshore and near-shore images, and where the nearshore data has varying shoreline features including its shape and position.
- **Realistic objects/obstacles:** If we imagine having a deep-learning based application that is reading all incoming satellite imagery in order to have a self-updating database of bathymetry profiles from around the globe, we need to

take into account the different features that can cause issues with the application's ability to make accurate predictions, including points where rivers connect to the sea, and the possibility of having 'alien' objects in the images such as wave-breakers or ships.

Wave simulation: As mentioned before, seafloors vary greatly based on their environment, ranging from sandy or muddy beaches, to rocky seafloors. This aspect also has an affect on how the waves develop, and how they react to their underlying bathymetry. Coupled with introducing these realistic features into the creation of synthetic bathymetries, the wave simulator should be configured (or developed) to resemble realistic wave conditions according to these different environments.

Dataset processing & extraction: Finally, when the data is extracted and the dataset is actually created, a method to extract sub-datasets based on different features (wave speeds, depth range, seafloor type) should be developed, in order to be able to work in a more controlled environment. This would help in answering many different possible questions and exploring different ideas, such as:

- Answering the question described in section 6.4, which is to use deep learning to find the depth limit where waves patterns are no longer affected by the underlying bathymetry.
- Developing separate deep learning models that deal with different types of bathymetries and waves.

With regards to the dataset, another idea that will be explored is using the image/data augmentation technique proposed by Bergsma et al. in [4] to increase the resolution of Sentinel-2's bands having a resolution lower than $10m$ (See section 3.1, table 2). If the bands with a resolution of $20m$ were to be augmented to have a $10m$ resolution, that would double the layers in the input data, and consequently would provide the deep learning models with double the temporal information (from $1.005s$ to $2.055s$), which should theoretically help the deep learning models in making more accurate estimations.

On the other hand, there are many ideas to be investigated concerning the deep learning part of the work. The first of which would be to run a **hyper-parameter search** experiment using the best-performing ranges found in section 7.3.2, in order to find other hyper-parameter values that can push the efficiency of the learning even further. Another way to improve the learning would be to use the **Neuromodulation** framework as proposed by Wilson et al. in [32]. This framework is inspired by biological neuromodulation, and it applies concepts from genetic algorithms to evolve the different weights and learning parameters at each layer and at each learning step, in a way that aims to improve the learning optimization algorithm (SGD

or Adam).

Moreover, from a real-application point-of-view, it would be very useful to perform a **predictive uncertainty study**, in order for the model to provide a confidence estimate along with its prediction. Being able to provide this information accurately is important for all types of deep learning applications, as presented in [26] for example. And in my case, such information is crucial when considering the possibility of coupling a deep learning model with other physics-based numerical models, in a data assimilation scheme for bathymetry inversion.

The last point concerning the deep learning part of the work is more related to the first approach I worked on during my internship, and it is based on the fact that we ended up with two different well-performing loss functions (MSLE, MAXERROR). The idea here is to explore **multi-objective optimization** [29], to train our model using multiple loss or objective functions at once instead of one, which could help our model to improve its predictive abilities even further.

Finally, and on a more personal note, this internship has been a valuable opportunity for me to widen my perspective and expand my skillset. From a computer science point-of-view, I was able to dig deeper into how deep neural networks work, by studying different types of architectures and how each can be utilized to perform different tasks. I got the chance to work on two different approaches using two state-of-the-art architectures, under the supervision of Dr. Dennis G WILSON, which was a truly valuable chance for me to expand and apply what I had already learned about deep neural networks, and to gain experience in using and analyzing the behavior of these advanced artificial intelligence algorithms.

Moreover, the coupling between two different domains of research into a single project made for a very interesting intellectual challenge. It introduced me to a lot of concepts and ideas that were new to me, and taught me how to adapt my way of thinking when dealing with multi-disciplinary projects. The work done during this internship can be considered as a starting point to a more in-depth and long-term project that will be carried on with in the near future. It allowed us - the team - to compare two different deep-learning-based approaches, to concretize our initial idea, to comprehend our constraints and limitations, and consequently to define the different paths we could take and explore in the future. In a broader context, with further research, the coupling between deep learning and the physics-based marine sciences could lead to many different advancements that can have a global impact.

Personally, I am genuinely thankful and proud to have had the chance to work on such a project, and to collaborate with and work under the supervision of such individuals. It was a great introduction to the research environment which I am hoping to continue working in.

References

- [1] R. Almar, E. W. Bergsma, P. Maisongrande, and L. P. M. de Almeida. Wave-derived coastal bathymetry from satellite video imagery: A showcase with pleiades persistent mode. *Remote Sensing of Environment*, 231:111263, 2019. ISSN 0034-4257. doi: <https://doi.org/10.1016/j.rse.2019.111263>. URL <http://www.sciencedirect.com/science/article/pii/S0034425719302822>.
- [2] M. Baawain, M. Gamal El-Din, and D. W. Smith. Artificial neural networks modeling of ozone bubble columns: Mass transfer coefficient, gas hold-up, and bubble size. *Ozone Science and Engineering*, 29:343–352, 10 2007. doi: 10.1080/01919510701549236.
- [3] E. Bergsma and R. Almar. Video-based depth inversion techniques, a method comparison with synthetic cases. *Coastal Engineering*, 138, August 2018. doi: 10.1016/j.coastaleng.2018.04.025.
- [4] E. Bergsma, R. Almar, and P. Maisongrande. Radon-augmented Sentinel-2 satellite imagery to derive wave-patterns and regional bathymetry. Submitted, 2019.
- [5] F. Bertini, O. Brand, S. Carlier, U. Del Bello, M. Drusch, R. Duca, V. Fernandez, C. Ferrario, M. Ferreira, C. Isola, V. Kirschner, P. Laberinti, M. Lambert, G. Mandorlo, P. Marcos, P. Martimort, S. Moon, P. Oldeman, M. Palomba, and J. Pineiro. Sentinel-2 esa’s optical high-resolution mission for gmes operational services. *ESA bulletin. Bulletin ASE. European Space Agency*, SP-1322, 03 2012.
- [6] E. Blayo, E. Cosme, M. Nodet, and A. Vidart. Introduction to data assimilation, 2011.
- [7] Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger. 3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation. *arXiv e-prints*, art. arXiv:1606.06650, Jun 2016.
- [8] F. Chollet et al. Keras. <https://keras.io>, 2015.
- [9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*, 2009.
- [10] A. Dertat. Applied deep learning - part 3: Autoencoders - towards data science, 2017. URL <https://towardsdatascience.com/>.

- [11] J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *J. Mach. Learn. Res.*, 12:2121–2159, July 2011. ISSN 1532-4435. URL <http://dl.acm.org/citation.cfm?id=1953048.2021068>.
- [12] Z. Fountas. Imperial college spiking neural networks for human-like avatar control in a simulated environment, 07 2019.
- [13] M. Ghotra and R. Dua. *Neural Network Programming with TensorFlow*. Packt Publishing, 2017. ISBN 9781788390392. URL <https://books.google.fr/books?id=7-JatAEACAAJ>.
- [14] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. *arXiv e-prints*, art. arXiv:1512.03385, Dec 2015.
- [15] V. Iglovikov and A. Shvets. TernaUSNet: U-Net with VGG11 Encoder Pre-Trained on ImageNet for Image Segmentation. *arXiv e-prints*, art. arXiv:1801.05746, Jan 2018.
- [16] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization, 2014. URL <http://arxiv.org/abs/1412.6980>. cite arxiv:1412.6980Comment: Published as a conference paper at the 3rd International Conference for Learning Representations, San Diego, 2015.
- [17] Q. V. Le, G. Brain, and G. Inc. A tutorial on deep learning part 1: Nonlinear classifiers and the backpropagation algorithm, 2015.
- [18] Q. V. Le, G. Brain, and G. Inc. A tutorial on deep learning part 2: Autoencoders, convolutional neural networks and recurrent neural networks, 2015.
- [19] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. *CoRR*, abs/1411.4038, 2014. URL <http://arxiv.org/abs/1411.4038>.
- [20] H. Mahmood. Gradient descent - towards data science, 2019. URL <https://towardsdatascience.com/>.
- [21] J. McCarthy. What is artificial intelligence?, 2004.
- [22] M. Nielsen. Neural networks and deep learning: Chapter 5, 2019. URL <http://neuralnetworksanddeeplearning.com/chap5.html>.
- [23] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. *arXiv e-prints*, art. arXiv:1505.04597, May 2015.

- [24] S. Ruder. An overview of gradient descent optimization algorithms. *CoRR*, abs/1609.04747, 2016. URL <http://arxiv.org/abs/1609.04747>.
- [25] S. Saha. A comprehensive guide to convolutional neural networks — the eli5 way, 2018. URL <https://towardsdatascience.com/>.
- [26] M. Segù, A. Loquercio, and D. Scaramuzza. A General Framework for Uncertainty Estimation in Deep Learning. *arXiv e-prints*, art. arXiv:1907.06890, Jul 2019.
- [27] P. Sharma. A comprehensive tutorial to learn convolutional neural networks from scratch (deeplearning.ai course #4), 2018. URL www.analyticsvidhya.com.
- [28] F. Shi, J. Kirby, B. Tehranirad, J. Harris, Y.-K. Choi, and M. Malej. Funwave-tvd fully nonlinear boussinesq wave model with tvd solver - documentation and user’s manual (version 3.0), 12 2016.
- [29] C. Silva Miranda and F. José Von Zuben. Multi-Objective Optimization for Self-Adjusting Weighted Gradient in Machine Learning Tasks. *arXiv e-prints*, art. arXiv:1506.01113, Jun 2015.
- [30] K. Simonyan and A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv e-prints*, art. arXiv:1409.1556, Sep 2014.
- [31] T. Tieleman and G. Hinton. Lecture 6.5—RmsProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.
- [32] D. G. Wilson, S. Cussat-Blanc, H. Luga, and K. Harrington. Neuromodulated Learning in Deep Neural Networks. *arXiv e-prints*, art. arXiv:1812.03365, Dec 2018.
- [33] W. Yao, Z. Zeng, C. Lian, and H. Tang. Pixel-wise regression using u-net and its application on pansharpening. *Neurocomputing*, 312:364 – 371, 2018. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2018.05.103>. URL <http://www.sciencedirect.com/science/article/pii/S0925231218307008>.